Алгоритмы синхронизации процессов

Тема 7

Алгоритмы синхронизации процессов

Содержание:

- race condition
- критическая секция
- алгоритмы синхронизации
- аппаратная поддержка взаимоисключения

Зачем нужна синхронизация

Для корректного взаимодействия процессов недостаточно только средств связи.
Необходимо:

- согласовывать доступ к общим данным
- предотвращать ошибки при параллельном выполнении
- Это достигается с помощью **алгоритмов синхронизации**.

Активности и атомарные операции

Активность — последовательность действий для достижения цели.
Активность состоит из **атомарных операций**.
Атомарная операция:

- выполняется как единое целое
- не может быть прервана

Пример активности:

1. приготовление бутерброда:
2. отрезать хлеб
3. намазать масло
4. положить колбасу

Interleaving (чередование)

При псевдопараллельном выполнении операций происходит чередование (interleaving).

Пример:
Активность P:

$a \rightarrow b \rightarrow c$

Активность Q:

$d \rightarrow e \rightarrow f$

Возможные варианты выполнения:

- a b c d e f
- a b d c e f
- a d b c e f
- d e f a b c

Детерминированность

Детерминированный набор программ

- всегда дает одинаковый результат

Недетерминированный набор

- результат зависит от порядка выполнения операций

Недетерминированность может привести к ошибкам.

Race Condition

Race condition (состояние гонки)
возникает, когда:

- несколько процессов используют общие данные
- результат зависит от порядка выполнения операций

Процессы состязаются за доступ к ресурсам.

Условия Бернстайна

Позволяют определить, можно ли выполнять программы параллельно.

Обозначения:

- $R(P)$ — переменные, которые программа читает
- $W(P)$ — переменные, которые программа изменяет

Параллельное выполнение безопасно, если:

- $W(P) \cap W(Q) = \emptyset$
- $W(P) \cap R(Q) = \emptyset$
- $R(P) \cap W(Q) = \emptyset$

Критическая секция

Критическая секция (critical section) — участок программы, где происходит доступ к общим ресурсам.

Правило:

в каждый момент времени **только один** процесс может находиться в критической секции.

Это называется **взаимоисключением** (mutual exclusion).

Структура процесса

Структура программы с критической секцией:

```
while(condition){

   entry section

   critical section

   exit section

   remainder section
}
```

Entry section — вход в критическую секцию
Exit section — выход
Remainder section — остальная часть программы

Требования к алгоритмам синхронизации

Хороший алгоритм должен:

1. работать без специальных аппаратных команд
2. не зависеть от скорости процессов
3. обеспечивать взаимоисключение
4. обеспечивать прогресс (progress)
5. обеспечивать ограниченное ожидание (bounded waiting)

Запрет прерываний

Простой метод:

запретить прерывания
critical section
разрешить прерывания

Недостатки:

- пользователь может заблокировать систему
- применяется только внутри ОС

Переменная-замок

Используется общая переменная:

```
lock = 0
```

Алгоритм:

```
while(lock);
lock = 1;

critical section

lock = 0
```

Недостаток:

- операции проверки и изменения **не атомарны**.

Строгое чередование

Используется переменная **turn**:
shared int turn

Процессы выполняют критическую секцию строго по очереди.
Недостаток:

- нарушается **условие прогресса**.

Алгоритм Петерсона

Работает для двух процессов.

Используются:
- массив готовности ready[ ]
- переменная turn

Преимущества:
- выполняет все требования
- обеспечивает взаимоисключение
- предотвращает тупики

Алгоритм булочной

Алгоритм для n процессов.

Идея:
как очередь в булочной.
Каждый процесс получает номер.
Процесс с **минимальным номером** выполняется первым.

Используются массивы:
choosing[ ]
number[ ]

Аппаратная поддержка

Процессоры имеют специальные атомарные команды.

Test-and-Set

- проверяет и устанавливает значение переменной.

Swap

- обменивает значения двух переменных.

Эти команды позволяют:

- упростить алгоритмы
- ускорить синхронизацию

Заключение

Основные понятия синхронизации:

- interleaving
- race condition
- критическая секция
- взаимоисключение

Основные алгоритмы:

- переменная-замок
- алгоритм Петерсона
- алгоритм булочной

Аппаратная поддержка (Test-and-Set, Swap) делает синхронизацию более эффективной.